Claude Opus 4.6 白盒代码审计方法论

Claude Opus 4.6 白盒代码审计方法论

——从"让 AI 找漏洞"到"设计审计决策体系"

> *不是教你用 AI 找漏洞，而是总结一套完整的审计决策框架：看什么、怎么看、看多深、何时停。*

一、为什么还需要方法论？

过去一年，大模型在代码审计领域进步非常明显。

GitHub 上涌现出大量基于 LLM 的审计工具，各种 Agent、Skill、Multi-Agent 框架层出不穷。我自己也基于 Claude Skill 实现了一套 code-audit skill，并持续在真实项目中使用和迭代。

在这个过程中有一个非常直观的感受：

> *模型能力已经足够强，但"如何使用模型"才是真正的瓶颈。*

很多人用 AI 审计的方式仍然停留在：

"帮我找这个项目的安全漏洞"

然后等待结果。

这种方式的问题很明显：

- 项目代码体量大，无法一次性进入上下文
- 单轮审计漏报率高（实际项目中可达 50%+）
- 容易出现"幻觉漏洞"
- 审计深度不可控

本质上，这是把 AI 当作搜索引擎在用。

但代码审计本身是一项高度依赖判断的工作，需要持续决策：

- 先看入口还是先看鉴权？
- 哪些模块优先级更高？
- 哪些链路值得深挖？
- 什么时候继续追踪，什么时候可以停止？

AI 可以分析代码，但不会自动替你设计审计策略。

二、这篇文章要解决什么问题？

这篇文章想讨论的，不是 prompt 技巧，也不是工具对比，而是更底层的一件事：

> *如何让大模型具备"审计思维"。*

在不断使用 Claude Opus 4.6 的过程中，我通过大量多轮对话和真实项目验证，逐步观察并总结出它在代码审计时的隐性逻辑，例如：

- 为什么会优先关注某些模块
- 如何从入口逐步收敛到高风险链路
- 在什么情况下会继续深挖
- 在什么情况下会停止分析

这些并不是简单的一次性回答，而是一个"逐步推进、不断收敛"的过程。

基于对 20 万行级别 Java 项目的实际审计，我把这些隐性的判断路径抽象成一套可复用的方法论，并逐步反哺到：

- code-audit skill 的设计
- agent 行为策略
- 多轮审计流程控制
- 后续框架能力提升

这套方法论的核心目标只有一个：

> *让 AI 不只是"看代码"，而是参与审计决策。*

它解决的不是"能不能找漏洞"的问题，而是：

- 先看什么
- 看多深
- 什么时候继续
- 什么时候收敛

当项目规模达到 10 万行以上时，这些决策直接决定审计质量和效率。

本文接下来会系统拆解从项目接手到报告输出的完整审计推进逻辑，以及每个阶段的判断依据与策略设计。

三、审计哲学：所有决策的源头

3.1 核心分析模型

所有安全漏洞本质上都是同一件事：**不可信的数据到达了危险的操作**。

```
Source（源）          用户可控的输入入口
      ↓
Propagation（传播）  数据流经的函数、过滤、转换
      ↓
Sink（汇）            SQL执行、命令执行、文件读写等危险操作
```

这个模型决定了审计的思考方式：**不逐文件遍历**（低效），而是**从外部入口向内部实现追踪数据流**，聚焦于用户输入最终到达的敏感操作点。

它也决定了审计的三个核心动作：

- 找到所有 Source（识别攻击面）
- 追踪 Propagation（验证中间是否有有效过滤）
- 确认是否有未过滤的路径到达 Sink（漏洞判定）

3.2 优先级排序公式

不是所有代码都值得同等审计。优先级的计算逻辑：

```
优先级 = (攻击面大小 × 潜在影响) / 利用复杂度
```

三个维度的权重：

维度	高权重	低权重
攻击面	未认证可达	内部接口
潜在影响	RCE、全库读取	信息收集
利用复杂度	单请求触发	需物理接触

第一条决策原则：永远优先审计认证链。 如果认证可被绕过，所有需认证才能触发的漏洞都会被放大为未认证漏洞。一个认证绕过能把 5 个 Medium 升级为 5 个 Critical。

四、五阶段处理流程

审计不是"搜一遍就完"，而是有明确阶段划分、精力分配和产出标准的工程过程。

```
Phase 1: 信息收集与攻击面识别      10% 精力
Phase 2: 并行模式匹配扫描          30% 精力
Phase 3: 关键路径手工审计          40% 精力
Phase 4: 漏洞验证与攻击链构建      15% 精力
Phase 5: 报告输出                   5% 精力
```

Phase 1：画地图，不是找漏洞

Phase 1 不搜漏洞。它的唯一目标是产出四样东西：

```
1. 技术栈画像 — 语言、框架、数据库、中间件
2. 模块地图   — 哪些模块负责什么功能
3. 攻击面清单 — 所有外部入口和出口
4. 安全机制   — 认证/授权/过滤/加密方案
```

为什么这一步不能跳过？ 因为后续所有 Agent 的方向、搜索模式、优先级排序，全部基于这张地图推导。没有地图就没有方向。

Phase 2：并行扫描，但有章法

Phase 2 启动多个 Agent 并行扫描，但不是随机搜索。Agent 的方向完全由 Phase 1 的攻击面地图推导（详见第五节）。

Phase 2 的核心产出是"高风险区域地图"，不是最终漏洞清单。发现率在 30-50% 是正常的——剩下的需要 Phase 3 的深度追踪才能确认。

Phase 3：逐行审计，按优先级

Phase 3 对 Phase 2 标记的高风险文件做逐行审计。文件优先级排序：

优先级	文件类型	决策依据
P0	认证过滤器 / Token 处理	认证绕过影响全系统
P0	白名单 / 未认证接口	直接暴露的攻击面
P0	核心业务入口（Sink 最密集的文件）	漏洞密度最高
P1	文件上传下载 / HTTP 出站工具类	路径遍历、SSRF 风险
P2	配置类、加密工具类	配置缺陷、密钥管理

逐行审计的处理逻辑：

```
对每个 P0/P1 文件：
  1. 读取完整结构（字段、方法列表）
  2. 从 public 方法开始，追踪每个参数的数据流
  3. 对每个分支判断：
     ├── 参数是否经过验证/过滤？→ 检查验证逻辑完整性
     ├── 参数是否到达 Sink？     → 记录完整路径
     └── 是否有绕过过滤的路径？  → 分析边界条件
  4. 记录：文件 → 行号 → 漏洞类型 → 完整数据流路径
```

关键决策：优先审计 Sink 聚合点。 如果项目有一个统一的 HTTP 出站工具类，审计这一个文件就覆盖了所有 SSRF 风险。

Phase 4：验证四步法

每个疑似漏洞必须过四关：

步骤	验证项	判定标准
1	数据流完整性	Source 到 Sink 中间无截断过滤
2	防护可绕过性	安全检查是否有遗漏的边界条件
3	前置条件可满足性	攻击者能否到达漏洞触发点
4	影响范围	利用后最大损害（RCE？数据泄露？提权？）

四步全过 → 确认漏洞。任一步不过 → 降级或排除。

五、Agent 并行策略：核心决策框架

这是方法论中处理逻辑最密集的部分。传统审计一个人串行做，AI 审计的核心优势是**多 Agent 并行**——但并行的方向不能随意定。

5.1 Agent 决策链

```
项目结构探测
    ↓
识别技术栈特征（语言 + 框架 + 功能模块 + 外部交互）
    ↓
推导攻击面（哪些地方可能有漏洞）
    ↓
按"可并行 + 不重叠"原则切分 Agent 任务
    ↓
执行 → 根据结果识别盲区 → 后续轮次补充
```

核心原则：Agent 的方向由攻击面决定，不是固定模板。 项目没有文件上传功能，就不设文件操作 Agent。项目没有数据源管理，就不设 JDBC 注入 Agent。

5.2 攻击面 → Agent 映射

探测发现的事实	推导出的审计方向	Agent 任务
Web 框架 + REST API	认证/授权链	Agent: 认证与授权
连接多种数据库	JDBC/SQL 注入	Agent: SQL 注入
有文件上传 + HTTP 出站	SSRF + 文件操作	Agent: 文件/SSRF
前后端分离 + 模板引擎	XSS/SSTI/RCE	Agent: 代码执行
插件系统/动态加载	类加载安全	Agent: 插件/数据源

5.3 切分的两个约束

约束 1：搜索模式互不重叠。 每个 Agent 有独占的 Grep 模式集，不重复搜索同一段代码。

约束 2：可完全并行执行。 Agent 之间零依赖，不需要等某个 Agent 完成后才启动另一个。

```
示例（Java Spring Boot 项目）：

Agent 1: SQL 注入
  搜索: ${}, Statement, "SELECT" +, String.format + SQL
  不搜索: JWT, Token, File, URL

Agent 2: 认证与授权
  搜索: JWT, Token, Session, Filter, @PreAuthorize, 白名单
  不搜索: SQL, File, HTTP

Agent 3: 文件操作 + SSRF
  搜索: MultipartFile, FileInputStream, Path, URL, HttpClient
  不搜索: JWT, SQL, ClassLoader

Agent 4: XSS/SSTI/RCE
  搜索: Runtime.exec, ProcessBuilder, ScriptEngine, eval
  不搜索: JWT, SQL, File

Agent 5: 数据源 + 插件
  搜索: JDBC, DriverManager, ClassLoader, Provider
  不搜索: JWT, Template, download
```

5.4 Agent 数量的决策因素

因素	判断标准
攻击面大小	功能越多 → Agent 越多
代码量	>100K LOC → 5+ Agent 并行加速
功能交叉度	交叉越多 → 切分越细
发现密度	某方向发现集中 → 专设 Agent 深挖

经验值：小型项目 2-3 个 Agent，中型 3-5 个，大型 5-9 个。

5.5 不同项目类型的策略差异

同一套决策框架，面对不同项目会推导出完全不同的 Agent 组合：

Go 微服务： 路由中间件 + SQL 注入 + 输入验证 + 并发安全。不需要反序列化、SSRF。

PHP WordPress 插件： SQL 注入 + 文件包含 + XSS + 权限检查。不需要 JDBC、JWT。

Android APK： 组件暴露 + 数据存储 + WebView + 网络安全 + 加密实现。不需要 SQL 注入、SSRF。

C/C++ 网络服务： 缓冲区溢出 + 格式化字符串 + 整数溢出 + 内存安全 + 竞态条件。不需要 XSS、JWT。

不变的是决策框架，变化的是具体组合。

六、多轮审计：轮次决策与终止策略

这是方法论中最关键的决策机制。大多数 AI 审计做一轮就结束，但实测数据显示单轮漏报率高达 60%。

6.1 每轮的目标函数不同

多轮审计不是"重复做同一件事"。每轮执行不同类型的分析，对应不同的目标函数：

轮次	目标函数	方法	发现类型
第一轮	max(覆盖面)	模式匹配（Grep）	表面可见：硬编码、未验证、配置缺陷
第二轮	max(深度)	数据流追踪 （逐行审计）	隐藏在数据流中：拼接链、协议注入、 权限缺失
第三轮	max(关联度)	跨模块交叉验证	组合才危险：IDOR+白名单、 加密体系系统性缺陷

```
第一轮 = 侦察    "哪些地方可能有漏洞？"  → 产出：高风险区域地图
第二轮 = 利用    "每个点真的能利用吗？"  → 产出：确认的漏洞 + 完整数据流
第三轮 = 关联    "漏洞之间有什么联系？"  → 产出：攻击链 + 架构级修复建议
```

6.2 为什么多轮能发现更多

原因 1：一个入口多条路径。 第一轮定位文件，第二轮展开所有数据流路径。一个文件可能有 6 条独立注入路径。

原因 2：加密需要生命周期追踪。 模式匹配找到"密钥在哪"，但密钥的生成→存储→传输→使用全流程，需要第二轮追踪才能发现 IV 全零、MD5 密码哈希等问题。

原因 3：跨模块关联需要全局视角。 Agent A 看到白名单条目认为是设计意图；Agent B 检查文件下载但没查认证。只有第三轮的交叉验证才能发现：白名单 + 下载 + ID 可枚举 = 未认证 IDOR。

原因 4：业务逻辑漏洞无法模式匹配。 代码"写得对"但语义错误的漏洞，Grep 搜不到。只有追踪完整业务流程才能识别。

6.3 轮次终止的决策规则

每轮结束后，强制回答三个问题：

```
Q1: 有没有计划搜索但没搜到的区域？
    YES → 需要下一轮（补盲区）
    NO  → 继续判断

Q2: 发现的入口点是否都追踪到了 Sink？
    NO  → 需要下一轮（追数据流）
    YES → 继续判断

Q3: 高风险发现之间是否可能存在跨模块关联？
    YES → 需要下一轮（交叉验证）
    NO  → 审计终止，输出报告
```

三个问题的答案全部为 NO → 终止。任一为 YES → 继续。

6.4 轮次不是固定数

轮次由项目特征和每轮结果动态决定：

项目规模	典型轮次	典型 Agent 总数
小型（<10K LOC）	1 轮	2-3 个
中型（10K-50K）	1-2 轮	3-5 个
大型（50K-200K）	2-3 轮	5-9 个
超大型（>200K）	2-4 轮	8-15 个

每轮的工具组合也不同：

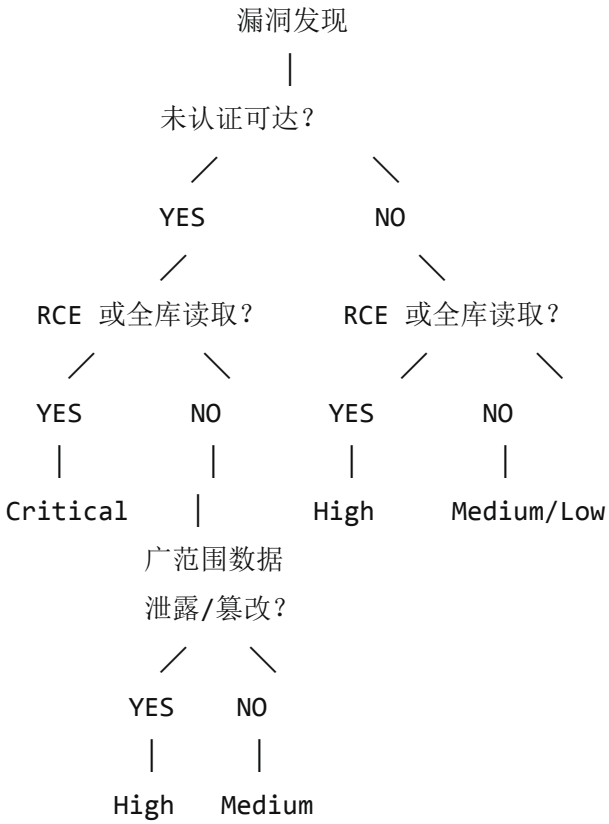
维度	第一轮	第二轮	第三轮
主要工具	Grep（模式搜索）	Read（逐行审计）	逻辑推理
Agent 数量	多（3-5 个并行）	中（2-3 个）	少（1 个）
发现特征	显而易见	隐藏在数据流中	需要组合理解

七、严重等级判定：三维决策模型

漏洞等级不靠感觉。用三维交叉评估：

```
严重等级 = f(可达性，影响范围，利用复杂度)
```

7.1 决策树



7.2 编号体系

使用 **C/H/M/L 前缀 + 序号**：

前缀	CVSS 3.1	含义
C	9.0-10.0	可直接导致系统沦陷
H	7.0-8.9	可造成重大损害
M	4.0-6.9	可造成一定损害
L	0.1-3.9	安全加固建议

编号排序策略：影响最广排最前 → 攻击链根漏洞优先 → 同模块漏洞相邻。

7.3 攻击链对等级的影响

规则：漏洞 A（认证绕过）消除漏洞 B 的认证前置条件
→ B 的"有效等级"按"未认证可达"重新评估

报告策略：
编号等级 = 独立等级（假设攻击者无其他漏洞）
攻击链部分 = 组合等级
好处：修复链中一环不影响剩余漏洞的独立评级

八、攻击链构建策略

8.1 构建方法

对每个 Critical/High 漏洞：

1. 确认前置条件（需要认证？什么权限？什么网络位置？）
2. 如果需要认证 → 检查是否有认证绕过可串联
3. 确认利用结果（信息泄露？代码执行？权限提升？）
4. 检查结果能否作为下一个漏洞的输入
5. 组合为完整攻击链，评估整体影响

8.2 典型攻击链模式

模式	链路	最终影响
认证绕过 → 功能滥用	Token 伪造 → 管理 API → 系统控制	全系统沦陷
认证绕过 → SSRF	JWT 伪造 → SSRF → 云元数据 → 获取凭据	云环境接管
认证绕过 → JDBC	Token 伪造 → 创建 H2 数据源 → RUNSCRIPT	服务器 RCE
信息泄露 → 认证绕过	密钥接口暴露 → 伪造 Token → 全系统访问	全系统沦陷
配置缺陷 → 数据窃取	CORS * → 恶意页面 → 跨域读取数据	用户数据泄露

九、技术栈识别与工具策略

9.1 四维识别框架

技术栈识别不是"看看用了什么框架"这么简单。它是整个审计方向的决策基础，需要识别四个维度：

```
语言类型  → 决定漏洞的代码表现形式（搜什么模式）
框架      → 决定安全机制的检查点（Filter? 中间件? 装饰器?）
功能模块  → 决定攻击面的广度（需要几个 Agent）
外部交互  → 决定边界审计的重点（入站重注入，出站重 SSRF）

四维叠加 → 得出具体的审计任务切分方案
```

同一种漏洞在不同语言中的搜索模式完全不同：

漏洞	Java	Python	Go
SQL 注入	Statement + 字符串拼接	cursor.execute(f"...")	fmt.Sprintf + db.Query
命令注入	Runtime.exec(input)	os.system(cmd)	exec.Command("sh","-c",cmd)
反序列化	readObject()	pickle.loads()	encoding/gob (较少风险)

不识别技术栈就开始搜索，等于在黑暗中射击。

9.2 面、线、点三层工具配合

```
Grep  → 面的搜索（快速定位高风险区域）
Read  → 线的追踪（数据流跟踪）
推理  → 点的验证（漏洞确认）
Agent → 并行加速（多维度同时扫描）
```

审计阶段	工具	用途
Phase 1	Glob, Bash	项目结构探索
Phase 2	Grep + 并行 Agent	快速定位潜在漏洞点
Phase 3	Read（逐行审计）	手工数据流追踪
Phase 4	逻辑推理	漏洞验证与攻击链构建
Phase 5	Write	结构化报告输出

十、关键决策原则速查

原则	决策依据
认证链优先	认证绕过放大所有其他漏洞，永远最先审计
追踪 Sink 聚合点	一个公共出口文件覆盖整个风险面
攻击面密度 > 代码量	5K 行的密码管理器可能比 100K 行的 CMS 更需深审
模式匹配是侦察不是结论	第一轮 Grep 的价值是画地图，不是出最终报告
每轮目标函数必须不同	广度→深度→关联，递进推进，重复搜索是浪费
跨模块漏洞最容易遗漏	单维度审计看不到组合问题，交叉验证是必要收尾
独立评级 + 组合评级	修复一个漏洞不影响其他漏洞的评级准确性
宁可多做一轮	多做一轮的成本远低于遗漏一个 Critical

十Opus 4.6 在这套方法论中的角色

方法论是框架，模型是执行引擎。Claude Opus 4.6 在这套方法论中承担的角色：

能力	对应方法论环节
多 Agent 并行	Phase 2 同时启动 5-9 个子任务，各聚焦一个漏洞域
深度代码理解	Phase 3 追踪参数从 Controller → Service → Mapper → SQL 的完整路径
跨文件推理	Phase 4 攻击链构建，同时理解认证模块、业务模块、数据源模块的关系
工具整合	Grep 做面的搜索、Read 做线的追踪、逻辑推理做点的验证

但模型只是工具。**没有方法论，再强的模型也只是在做高级 Grep。** 方法论决定了看什么、怎么看、看几轮、什么时候停。

总结：方法论的六个支柱

1. 污点分析驱动 — Source→Propagation→Sink，所有决策围绕数据流
2. 攻击面导向 — 从项目功能推导审计方向，不套模板
3. Agent 并行策略 — 搜索模式互不重叠，完全并行执行
4. 多轮递进 — 广度→深度→关联，每轮目标函数不同
5. 攻击链思维 — 组合低危漏洞构建高危攻击路径
6. 系统性决策 — 三维等级评估、轮次终止规则、优先级排序公式

这套方法论不限于 Claude，也不限于 Java。它适用于任何语言、任何框架、任何 AI 模型的白盒代码安全审计。